

Multi Source Candidate Data Transformer

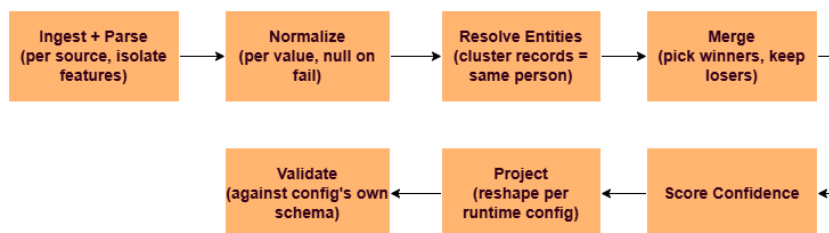
Hussain Haidary | hussain.haidary452@gmail.com

Problem:

Candidate data is obtained from various sources simultaneously including structured (i.e. ATS JSON, Recruiter CSV) and unstructured (Recruiter comments, GitHub). The same candidate could exist in multiple sources with different values; any particular source can be non-existent, blank or corrupted.

The objective is to generate a single standardized profile for every candidate in normalized format, containing unique values and a level of confidence in the value.

Pipeline:



Canonical Schema:

candidate_id, full_name, emails[], phones[], location{city, region, country}, links{linkedin, github, portfolio, other[]}, headline, years_experience, skills[{name, confidence, source[]}], experience[{company, title, start, end, summary}], education[{institution, degree, field, end_year}], provenance[{field, source, method}], overall_confidence.

Normalized Formats: (Every normalizer will return a value or null)

1. Phones: E.164 (libphonenumber; invalid rejected, not coerced)
2. Countries: ISO-3166 alpha-2
3. Dates: YYYY-MM (bare year: -01; "Present": null = ongoing)
4. Skills: canonical names (alias table; unknowns kept, never dropped)

Conflict Resolution and Confidence:

Within the merged candidate cluster, data conflicts are resolved strictly at the field level. By default, the winning value is determined by a source-trust hierarchy (ATS > CSV > GitHub > Notes). However, agreement constitutes evidence: if a value is corroborated by two independent sources, it overrides a single value from a higher-trusted source. Array fields like emails, phones, and skills are simply combined. For complete auditability, losing values are never dropped in silence but rather stored in the provenance log. Information from GitHub is merely an enrichment layer by providing links and skills for a resolved candidate without any entity matching being performed.

Data reliability is quantified dynamically. Field-level confidence is calculated as $\text{source_trust} \times \text{corroboration_bonus} \times \text{normalization_success}$, rewarding corroborated and successfully standardized data (like an E.164 phone number). The overall_confidence score is a transparent aggregate that heavily weights core identifying fields (Name, Email, Phone), ensuring that baseline profile identity is prioritized over peripheral skills.

Runtime custom-output configuration:

A JSON/YAML config selects a subset of fields, renames from a canonical path ('email[0]' -> 'primary_email', 'skills[].name' -> flat 'string[]'), and chooses missing-value behavior: `null` | `omit` | `error`. Output is validated against a schema derived from the config – the output's contract is whatever the config asked for, not the canonical schema. The config itself is validated before the pipeline runs, so misconfiguration fails fast and explainably.

Edge cases:

1. Ghost candidate (name, no email/phone): No strong key, so it's quarantined rather than risking a false merge.
2. Format issues: '(xxx) xxx-xxxx' against '+91 xxx xxx xxxx' both normalize to one E.164; structurally invalid numbers are rejected.
3. Config requests an unknown field: Caught by config validation before run starts, with a clear error message.
4. Malformed source: Isolated inside its own parser; logged as a warning, run continues on the remaining sources.

Deliberately Descope:

1. LLM based extraction (deterministic rules instead, for reproducibility)
2. Live OAuth (GitHub via public API only)
3. Persistent Storage (in memory)
4. Fuzzy/ML name dedupe (name stays a weak signal)